



МИНОБРНАУКИ РОССИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технологический университет
«СТАНКИН»
(ФГАОУ ВО «МГТУ «СТАНКИН»)

Институт информационных технологий
Кафедра информационных технологий и вычислительных систем

Образовательная программа 09.03.01
«Информатика и вычислительная техника»

Дисциплина «Структуры и алгоритмы обработки данных»

Отчет
по лабораторной работе № 1

Выполнил:

студент гр. ИДБ-25-02

(дата)

(подпись)

Козицкий А. В.

Принял:

к.т.н., доцент

(дата)

(подпись)

Ковалев И.А.

Москва 2026

Введение

Экспериментальное исследование сложности алгоритмов

Цель лабораторной работы — реализовать несколько алгоритмов, измерить время их работы на входных данных разного размера и сравнить экспериментальные результаты с теоретической сложностью.

Задание 1

Проверка наличия элемента в массиве

Идея алгоритма простая: нужно последовательно пройти по массиву и сравнить каждый элемент с искомым значением.

Если элемент найден — вернуть True.

Если массив закончился — вернуть False.

При выполнении заданий используется файл func.py содержащий основные функции для выполнения задания (такие как генерация массива, либо замер времени выполнения функции)

func.py

```
import time
import random

def generate_array(n):
    arr = []
    for i in range(n):
        arr.append(random.randint(0, 10000))
    return arr

def measure_time1(func, data1, data2):
    start = time.perf_counter()
    func(data1, data2)
    end = time.perf_counter()
    return end - start

def measure_time2(func, data):
    start = time.perf_counter()
    func(data)
    end = time.perf_counter()
    return end - start
```

l.py

```
import func
import random

def find_element(arr, x):
    for i in arr:
        if i == x:
            return True
    return False
```

```

if __name__ == "__main__":
    sizes = [100, 1000, 5000, 10_000]
    for n in sizes:
        arr = func.generate_array(n)
        t = func.measure_time1(find_element, arr, random.randint(0, 10_000))
        print(n, t)

```

Код проходит по каждому элементу массива и возвращает True если найден элемент x

```

100 5.799998689326458e-06
1000 2.1100000594742596e-05
5000 5.740000051446259e-05
10000 0.0004085000000486616

```

Результат работы:

Задание 2

Поиск второго максимального элемента

Алгоритм должен найти второе по величине число в массиве.

Идея алгоритма:

1. хранить два значения — максимальное и второе максимальное (т.е. на первой итерации $max1 = arr[0]$, $max2 = arr[0]$)
2. пройти по массиву
3. обновлять значения при необходимости

2.py

```

import func

def find_two_max(arr):
    max1 = arr[0]
    max2 = arr[0]
    for i in arr:
        if max1 < i:
            max1 = i

    for i in arr:
        if max2 < i and i != max1:
            max2 = i
    return [max1, max2]

if __name__ == "__main__":
    sizes = [100, 1000, 5000, 10_000]
    for n in sizes:
        arr = func.generate_array(n)
        t = func.measure_time2(find_two_max, arr)
        print(n, t)

```

Код сначала находит максимальный элемент массива, а после ищет максимальный элемент массива который не равен первому максимуму. Возвращает список из первого и второго максимума

Результат работы:

```
100 8.600000001024455e-06
1000 3.590000051190145e-05
5000 0.0001754999938637484
10000 0.000358400005503185
```

Задание 3

Бинарный поиск

Бинарный поиск используется для поиска элемента в **отсортированном массиве**.

Алгоритм работает следующим образом:

1. выбирается середина массива
2. если элемент найден — поиск завершён
3. если элемент меньше — поиск продолжается слева
4. если больше — справа

3.py

```
import func
import random

def bin_find(arr):
    arr.sort()
    a = random.choice(arr)
    while True:
        z = len(arr)
        if arr[z//2] == a:
            return True
        elif a > arr[z//2]:
            arr = arr[z//2 + 1:]
        else:
            arr = arr[:z//2]

if __name__ == "__main__":
    sizes = [100, 1000, 5000, 10_000]
    for n in sizes:
        arr = func.generate_array(n)
        t = func.measure_time2(bin_find, arr)
        print(n, t)
```

Код смотрит в середину отсортированного массива, если искомый элемент больше медианного то алгоритм повторяется во второй части массива (если меньше, то в первой)

```
100 2.0399998902576044e-05
1000 0.00011009999980160501
5000 0.0017385000010108342
10000 0.0013139999991835793
```

Результат работы:

Задание 4

Построение таблицы умножения

Идея алгоритма:

1. внешний цикл проходит по строкам
2. внутренний цикл проходит по столбцам
3. для каждой пары чисел вычисляется произведение
- 4.

Например для $n = 3$:

1 2 3

2 4 6

3 6 9

4.py

```
import func

def tab_mult(n):
    for i1 in range(1, n+1):
        for i2 in range(1, n+1):
            print(i1 * i2, end = " ")
        print()

if __name__ == "__main__":
    sizes = [2, 3, 4, 10]
    for n in sizes:
        t = func.measure_time2(tab_mult, n)
        print(n, t)
```

Код двойным циклом проходит по каждому элементу из диапазона n , составляя таблицу умножения

Результат работы:

```

1 2
2 4
2 0.00019520000023476314
1 2 3
2 4 6
3 6 9
3 0.00016490000052726828
1 2 3 4
2 4 6 8
3 6 9 12
4 8 12 16
4 0.00022059999901102856
1 2 3 4 5 6 7 8 9 10
2 4 6 8 10 12 14 16 18 20
3 6 9 12 15 18 21 24 27 30
4 8 12 16 20 24 28 32 36 40
5 10 15 20 25 30 35 40 45 50
6 12 18 24 30 36 42 48 54 60
7 14 21 28 35 42 49 56 63 70
8 16 24 32 40 48 56 64 72 80
9 18 27 36 45 54 63 72 81 90
10 20 30 40 50 60 70 80 90 100
10 0.000663499999063788

```

Задание 5

Провести замеры алгоритмической и пространственной сложности любой из сортировок

1. Найти любую реализацию сортировки(Сортировка вставками, Быстрая сортировка и т.п.)
2. Написать код измеряющий алгоритмическую сложность, а так же пространственную (реализовать самостоятельно)
3. Оформить результаты в виде markdown таблицы в Readme файла репозитория

buble_sort.py

```

import tracemalloc
import func

def buble_sort(arr):
    tracemalloc.start()
    n = len(arr)
    for i in range(n):
        f = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

```

```

        f = True
    if not f:
        break
    current, peak = tracemalloc.get_traced_memory()
    print(f"Текущая память: {current} байт, пиковая: {peak} байт")

if __name__ == "__main__":
    sizes = [100, 1000, 5000, 10_000]
    for n in sizes:
        arr = func.generate_array(n)
        t = func.measure_time2(buble_sort, arr)
        print(n, t)

```

В коде используется сортировка пузырьком

После каждой сортировки выводится сколько времени она заняла и сколько памяти

Результат работы:

```

Текущая память: 32 байт, пиковая: 32 байт
100 0.0021505999993678415
Текущая память: 40232 байт, пиковая: 40232 байт
1000 0.066186300000048142
Текущая память: 198144 байт, пиковая: 235008 байт
5000 9.3432494000000786
Текущая память: 400352 байт, пиковая: 591968 байт
10000 27.40432619999956

```

Github репозиторий данной лабораторной:

https://github.com/Stafferz/laboratory_work_on_structures_and_algorithms/tree/master/lab1